

## Градиентный бустинг - 2

Александр Безносиков

ФПМИ МФТИ

02 декабря 2025



# Повторение градиентного бустинга

Функционал качества  $\mathcal{L}(y, \hat{y}) = \sum_{i=1}^n \ell(y^i, \hat{y}(x^i)) \rightarrow \min_{\hat{y}}$ ,  
где  $\ell(y, z)$  — кусочно дифференцируемая функция потерь.  
Строим композицию в виде  $\hat{y}_T(x) = \sum_{t=0}^T \gamma_t g_t(x)$ .

**Построение новой модели на шаге  $t$ :**

- ① Вычислить градиенты по обучающей выборке:

$$\tilde{e}_t = (\nabla_s \ell(y^i, s)|_{s=\hat{y}_{t-1}(x^i)})_{i=1}^n$$

- ② Обучить новую модель по MSE по выборке  $(x^1, -\tilde{e}_t^1), \dots, (x^n, -\tilde{e}_t^n)$ :

$$g_t(x) = \operatorname{argmin}_{g \in \mathcal{G}} \sum_{i=1}^n (g(x^i) + \tilde{e}_t^i)^2$$

- ③ Подобрать коэффициент при  $g_t$ :

$$\tilde{\gamma}_t = \operatorname{argmin}_{\gamma \in \mathbb{R}} \sum_{i=1}^n \ell(y^i, \hat{y}_{t-1}(x^i) + \gamma g_t(x^i))$$

- ④ Добавить модель к композиции:

$$\hat{y}_t(x) = \hat{y}_{t-1}(x) + \eta \tilde{\gamma}_t g_t(x)$$

# Градиентный бустинг над деревьями

Решающее дерево разбивает все пространство на *непересекающиеся области*, в которых его ответ равен константе:

$$g_t(x) = \sum_{k=1}^{n_t} g_{tk} \cdot \mathbb{I}_{x \in R_k}$$

где  $k = 1, \dots, n_t$  — индексы листьев,  $g_{tk}$  — значения в листьях,  $R_k$  — соответствующие области разбиения:  $\bigsqcup_{k=1}^{n_t} R_k = \mathcal{X}$ .

На  $t$ -й итерации композиция обновляется как:

$$\hat{y}_t(x) = \hat{y}_{t-1}(x) + \gamma_t \sum_{k=1}^{n_t} g_{tk} I\{x \in R_k\} = \hat{y}_{t-1}(x) + \sum_{k=1}^{n_t} \gamma_t g_{tk} I\{x \in R_k\}$$

**Замечание:** Добавление в композицию дерева с  $n_t$  листьями равносильно добавлению  $n_t$  базовых моделей, представляющих собой предикаты вида  $\mathbb{I}_{x \in R_k}$ . Если вместо общего  $\gamma_t$  будет свой  $\gamma_{tk}$  при каждом предикате, то можно подобрать его так, чтобы повысить качество композиции.

# Перенастройка в листьях

## Схема:

- Обучим дерево  $g_t \Rightarrow$  структура дерева задана
- Сделаем перенастройку в листьях обученного дерева

Тогда потребность в  $\gamma_t$  и  $g_{tk}$  отпадает:

$$\sum_{i=1}^n \ell \left( y^i, \hat{y}_{t-1}(x^i) + \sum_{k=1}^{n_t} \gamma_{tk} \cdot I\{x \in R_k\} \right) \rightarrow \min_{\{\gamma_{tk}\}_{k=1}^{n_t}}$$

Т.к. области разбиения  $R_k$  не пересекаются, задача распадается на  $n_t$  независимых подзадач:

$$\gamma_{tk} = \underset{\gamma}{\operatorname{argmin}} \sum_{x^i \in R_k} \ell(y^i, \hat{y}_{t-1}(x^i) + \gamma), \quad k = 1, \dots, n_t$$

В некоторых случаях оптимальные  $\gamma_{tk}$  можно найти аналитически — например, для MAE и MSE.

# История градиентного бустинга

- 1990: **AdaBoost**

Появился в процессе исследования вопроса о том, можно ли из многих слабых и простых моделей получить одну сильную.

- 1999: **Gradient Boosting**

- 2014: **XGBoost**

Стартовал как исследовательский проект Тяньцзи Чена. После победы в Higgs Machine Learning Challenge стал популярным в сообществе Kaggle.

- 2017, январь: **LightGBM**

Microsoft разработал "лёгкую" версию бустинга с оптимизациями для больших данных.

- 2017, июль: **CatBoost**

Яндекс представил свою версию с улучшенной обработкой категориальных признаков.

# Пространство $\mathbb{R}^n$

При построении градиентного бустинга мы сначала рассмотрели градиентный спуск в пространстве  $\mathbb{R}^n$ . Приведем соответствие обозначений.

## Объекты в $\mathbb{R}^n$ :

- $s \in \mathbb{R}^n$
- $\Delta s \in \mathbb{R}^n$
- $\mathcal{L}(y, s) = \sum_{i=1}^n \ell(y^i, s^i)$
- $\mathcal{L}(y, \hat{y}) = \sum_{i=1}^n \ell(y^i, \hat{y}(x^i))$

## Их значения в бустинге:

- $(\hat{y}_{t-1}(x^i))_{i=1}^n$
- $(g_t(x^i))_{i=1}^n$
- $\tilde{e} = \nabla_s \mathcal{L}(y, s)$
- $\tilde{e} = \nabla_s \mathcal{L}(y, \hat{y})$

# Построение $g_t$

Задача оптимизации при построении новой базовой модели

$$g_t(x) = \operatorname{argmin}_{g \in \mathcal{G}} \sum_{i=1}^n (g(x^i) + \tilde{e}_t^i)^2$$

В терминах пространства  $\mathbb{R}^n$  она выглядит так:

$$\begin{aligned} \operatorname{argmin}_{\Delta s \in \mathbb{R}^n} \sum_{i=1}^n (\Delta s^i + e^i)^2 &= \operatorname{argmin}_{\Delta s \in \mathbb{R}^n} \sum_{i=1}^n ((\Delta s^i)^2 + 2e^i \Delta s^i + (e^i)^2) \\ &= \operatorname{argmin}_{\Delta s \in \mathbb{R}^n} \sum_{i=1}^n \left( \frac{1}{2} (\Delta s^i)^2 + e^i \Delta s^i \right) \end{aligned}$$

**Вопрос:** Почему решаем именно такую задачу?

# Построение $g_t$

Задача оптимизации при построении новой базовой модели

$$g_t(x) = \operatorname{argmin}_{g \in \mathcal{G}} \sum_{i=1}^n (g(x^i) + \tilde{e}_t^i)^2$$

В терминах пространства  $\mathbb{R}^n$  она выглядит так:

$$\begin{aligned} \operatorname{argmin}_{\Delta s \in \mathbb{R}^n} \sum_{i=1}^n (\Delta s^i + e^i)^2 &= \operatorname{argmin}_{\Delta s \in \mathbb{R}^n} \sum_{i=1}^n ((\Delta s^i)^2 + 2e^i \Delta s^i + (e^i)^2) \\ &= \operatorname{argmin}_{\Delta s \in \mathbb{R}^n} \sum_{i=1}^n \left( \frac{1}{2} (\Delta s^i)^2 + e^i \Delta s^i \right) \end{aligned}$$

**Вопрос:** Почему решаем именно такую задачу?

Попробуем оптимизировать непосредственно функционал ошибки, рассмотрев его приближение по ф-ле Тейлора

# Оптимизация функционала ошибки

$$\begin{aligned}\mathcal{L}(y, s + \Delta s) &= \sum_{i=1}^n \ell(y^i, s^i + \Delta s^i) \approx \\ &\approx \sum_{i=1}^n \left[ \underbrace{\ell(y^i, s^i)}_{\text{не зав. от } \Delta s^i} + \underbrace{\Delta s^i \cdot \nabla_s \ell(y^i, s^i)}_{=e^i} + \frac{1}{2} (\Delta s^i)^2 \cdot \underbrace{\nabla_s^2 \ell(y^i, s^i)}_{=:h^i} \right].\end{aligned}$$

Тогда получили задачу оптимизации:

$$\operatorname{argmin}_{\Delta s \in \mathbb{R}^n} \sum_{i=1}^n \left[ e^i \Delta s^i + \frac{1}{2} h^i (\Delta s^i)^2 \right]$$

В обычном градиентном бустинге ранее получали:

$$\operatorname{argmin}_{\Delta s \in \mathbb{R}^n} \sum_{i=1}^n \left[ e^i \Delta s^i + \frac{1}{2} (\Delta s^i)^2 \right]$$

# eXtreme Gradient Boosting (XGBoost)

Обычный градиентный бустинг

$$\sum_{i=1}^n \left( \tilde{e}^i g(x^i) + \frac{1}{2} g^2(x^i) \right) \rightarrow \min_{g \in \mathcal{G}}$$

XGBoost

$$\sum_{i=1}^n \left( \tilde{e}^i g(x^i) + \frac{1}{2} h^i g^2(x^i) \right) \rightarrow \min_g$$

Вопрос: Что это нам даёт?

# eXtreme Gradient Boosting (XGBoost)

Обычный градиентный бустинг

$$\sum_{i=1}^n \left( \tilde{e}^i g(x^i) + \frac{1}{2} g^2(x^i) \right) \rightarrow \min_{g \in \mathcal{G}}$$

XGBoost

$$\sum_{i=1}^n \left( \tilde{e}^i g(x^i) + \frac{1}{2} h^i g^2(x^i) \right) \rightarrow \min_g$$

**Вопрос:** Что это нам даёт?

XGBoost использует аппроксимацию **второго порядка**, учитывая информацию о вторых производных ( $h^i$ ), в то время как обычный градиентный бустинг предполагает постоянную кривизну функции по всем направлениям (отбрасывает  $h^i$ ).

# Регуляризация в XGBoost

Дерево  $g(x)$  можно описать формулой:

$$g(x) = \sum_{k=1}^{n_I} \gamma_k \cdot \mathbb{I}_{x \in R_k}$$

где  $k = 1, \dots, n_I$  — индексы листьев,  $\gamma_k$  — значения в листьях,  $R_k$  — соответствующие области разбиения:  $\bigcup_{k=1}^{n_I} R_k = \mathcal{X}$

**Вопрос:** От чего зависит сложность  $g(x)$ ?

# Регуляризация в XGBoost

Дерево  $g(x)$  можно описать формулой:

$$g(x) = \sum_{k=1}^{n_l} \gamma_k \cdot \mathbb{I}_{x \in R_k}$$

где  $k = 1, \dots, n_l$  — индексы листьев,  $\gamma_k$  — значения в листьях,  $R_k$  — соответствующие области разбиения:  $\bigcup_{k=1}^{n_l} R_k = \mathcal{X}$

**Вопрос:** От чего зависит сложность  $g(x)$ ?

- 1 Число листьев  $n_l$   
(Чем больше листьев, тем сложнее разделяющая поверхность)
- 2 Норма коэффициентов в листьях  $\|g\|_2^2 = \sum_{k=1}^{n_l} \gamma_k^2$   
(Чем дальше коэффициенты от нуля, тем сильнее  $g$  влияет на ответ)

# Регуляризация в XGBoost

Дерево  $g(x)$  можно описать формулой:

$$g(x) = \sum_{k=1}^{n_I} \gamma_k \cdot \mathbb{I}_{x \in R_k}$$

где  $k = 1, \dots, n_I$  — индексы листьев,  $\gamma_k$  — значения в листьях,  $R_k$  — соответствующие области разбиения:  $\bigcup_{k=1}^{n_I} R_k = \mathcal{X}$

**Вопрос:** Как сложность  $g(x)$  влияет на итоговый ответ?

# Регуляризация в XGBoost

Дерево  $g(x)$  можно описать формулой:

$$g(x) = \sum_{k=1}^{n_I} \gamma_k \cdot \mathbb{I}_{x \in R_k}$$

где  $k = 1, \dots, n_I$  — индексы листьев,  $\gamma_k$  — значения в листьях,  $R_k$  — соответствующие области разбиения:  $\bigcup_{k=1}^{n_I} R_k = \mathcal{X}$

**Вопрос:** Как сложность  $g(x)$  влияет на итоговый ответ?

Чем сложнее  $g(x)$ , тем меньше ошибок мы делаем и тем больше вероятность переобучения.

**Вопрос:** Как ограничить сложность  $g(x)$ ?

# Регуляризация в XGBoost

Дерево  $g(x)$  можно описать формулой:

$$g(x) = \sum_{k=1}^{n_I} \gamma_k \cdot \mathbb{I}_{x \in R_k}$$

где  $k = 1, \dots, n_I$  — индексы листьев,  $\gamma_k$  — значения в листьях,  $R_k$  — соответствующие области разбиения:  $\bigcup_{k=1}^{n_I} R_k = \mathcal{X}$

**Вопрос:** Как сложность  $g(x)$  влияет на итоговый ответ?

Чем сложнее  $g(x)$ , тем меньше ошибок мы делаем и тем больше вероятность переобучения.

**Вопрос:** Как ограничить сложность  $g(x)$ ?

Добавим регуляризаторы, штрафующие за оба вида сложности:

$$\sum_{i=1}^n \left( \tilde{e}^i g(x^i) + \frac{1}{2} h^i g^2(x^i) \right) + \mu n_I + \frac{\lambda}{2} \sum_{k=1}^{n_I} \gamma_k^2 \longrightarrow \min_{g \in \mathcal{G}}$$

## Переподбор в листьях дерева

Пусть дерево построено, найдем оптимальные ответы в листьях:

$$\begin{aligned} & \sum_{i=1}^n \left( \tilde{e}^i g(x^i) + \frac{1}{2} h^i g^2(x^i) \right) + \mu n_l + \frac{\lambda}{2} \sum_{k=1}^{n_l} \gamma_k^2 = \\ &= \sum_{k=1}^{n_l} \left[ \left( \sum_{i \in R_k} \tilde{e}^i \right) \gamma_k + \frac{1}{2} \left( \sum_{i \in R_k} h^i + \lambda \right) \gamma_k^2 + \mu \right] = \\ &= \sum_{k=1}^{n_l} \left[ E_k \gamma_k + \frac{1}{2} (H_k + \lambda) \gamma_k^2 + \mu \right] \longrightarrow \min_{\gamma_k} \end{aligned}$$

- Задача разбивается на  $n_l$  независимых подзадач по листьям
- Оптимальное значение для каждого листа:

$$\gamma_k^* = \frac{-E_k}{H_k + \lambda} \quad (\text{оптимальный ответ в листьях})$$

# Перепо́дбор в листьях дерева

Оптимальные коэффициенты в листьях:

$$\gamma_k^* = \frac{-E_k}{H_k + \lambda},$$

где:

$$E_k = \sum_{i \in R_k} \tilde{e}^i, \quad H_k = \sum_{i \in R_k} \tilde{h}^i$$

Ошибка дерева с оптимальными коэффициентами в листьях:

$$\begin{aligned} \mathcal{L}^*(y, g) &= \sum_{k=1}^{n_I} \left[ E_k \gamma_k^* + \frac{1}{2} (H_k + \lambda) (\gamma_k^*)^2 + \mu \right] \\ &= -\frac{1}{2} \sum_{k=1}^{n_I} \frac{E_k^2}{H_k + \lambda} + \mu n_I \end{aligned}$$

# Обучение дерева в XGBoost

Функционал  $\mathcal{L}^*(y, g)$  для заданной структуры дерева  $g$  вычисляет минимальную ошибку, которую можно получить, подобрав оптимальные ответы в листьях:

$$\mathcal{L}^*(y, g) = \sum_{k=1}^{n_l} \left( -\frac{1}{2} \frac{E_k^2}{H_k + \lambda} + \mu \right) = \sum_{k=1}^{n_l} H(X_k)$$

Свойства  $H(X)$  как критерия информативности:

- Зависит только от выборки  $X$ , попавшей в вершину
- Уменьшается, когда объекты  $X$  лучше предсказываются константой  $\gamma$  (при оптимальном выборе  $\gamma$ )
- $H(X_k)$  характеризует ошибку предсказания в листе  $R_k$

**Вывод:**  $H(X)$  можно использовать как критерий информативности при построении дерева.

# Обучение дерева в XGBoost

Выбираем разбиение  $H(x^j < t)$  в вершине  $X_m$  путем решения:

$$\mathcal{L}(X_m, j, t) = H(X_m) - H(X_L) - H(X_R) \rightarrow \max_{j, t}$$

где  $X_L$  и  $X_R$  — левая и правая дочерние вершины, а информативность:

$$H(X) = -\frac{1}{2} \frac{(\sum_{i \in X} \tilde{e}^i)^2}{\sum_{i \in X} \tilde{h}^i + \lambda} + \mu$$

Структура дерева выбирается для минимизации функции потерь.

**Критерий останковки:** вершина становится листом, если лучшее разбиение даёт  $\mathcal{L} < 0$  (увеличивает ошибку  $\mathcal{L}^*(g)$ )

**Замечание:** В отличие от классических критериев

$H_{\text{classic}}(X) = \min_{c \in Y} \frac{1}{|X|} \sum_{x^i \in X} \ell(y^i, c)$ , в  $\mathcal{L}$  нет нормировки на размеры дочерних вершин, т.к. здесь  $H(X)$  зависит от размера классов

# Особенности XGBoost

## 1 Использование вторых производных

Базовая модель приближает направление, посчитанное с учетом вторых производных функции потерь.

## 2 Регуляризация

Штрафы за количество листьев и за норму коэффициентов.

## 3 Критерий информативности

При построении дерева используется критерий информативности, зависящий от оптимального вектора сдвига

## 4 Критерий останова

Критерий останова при обучении дерева также зависит от оптимального сдвига.

## 5 Дополнительные особенности

- Умная обработка пропущенных значений
- Аппроксимация порога  $t$

# LightGBM

**Вопрос:** Как можно интерпретировать большое значение антиградиента  $\tilde{e}^i$  для некоторого объекта?

# LightGBM

**Вопрос:** Как можно интерпретировать большое значение антиградиента  $\tilde{e}^i$  для некоторого объекта?

## Основная идея

Если антиградиент  $\tilde{e}^i$  большой на некотором объекте, то этот объект важен и данный  $\tilde{e}^i$  внесёт значительный вклад в уменьшение итоговой ошибки.

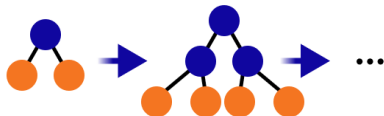
⇒ Используем  $\tilde{e}^i$  как вес объекта  $x^i$ .

## Алгоритм поиска разбиения в вершине дерева:

- 1 Выбираем объекты с наибольшими значениями  $\tilde{e}^i$
- 2 Добавляем к ним случайную выборку объектов с малыми  $\tilde{e}^i$
- 3 Используем только эту комбинированную выборку для нахождения оптимального разбиения

# Сравнение структуры обычного дерева и LightGBM

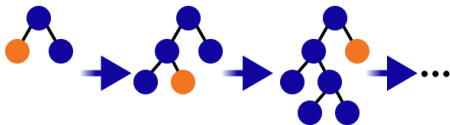
## Обычное решающее дерево



Дерево строится по уровням.

Для каждой вершины в уровне находим оптимальное разбиение.

## LightGBM



Среди всех листов дерева находим лист, который оптимальнее всего разделить на этой итерации.

Полученное дерево может быть очень несбалансированным.

# Особенности LightGBM

① **Особая структура деревьев:** Leaf-wise рост вместо level-wise

② **Градиентно-взвешенный отбор объектов**

- Приоритет объектам с большими градиентами
- Случайная подвыборка объектов с малыми градиентами

③ **Exclusive Feature Bundling (EFB)**

Признаки, которые редко бывают одновременно ненулевыми на одном объекте, заменяются на один признак.

④ **Поиск порогов по гистограммам**

- Дискретизация признаков на бины
- Рассмотрение только границ бинов как кандидатов в пороги

**Вопрос:** В каких задачах LightGBM будет справляться наилучшим образом, с учётом таких особенностей?

# Особенности LightGBM

① **Особая структура деревьев:** Leaf-wise рост вместо level-wise

② **Градиентно-взвешенный отбор объектов**

- Приоритет объектам с большими градиентами
- Случайная подвыборка объектов с малыми градиентами

③ **Exclusive Feature Bundling (EFB)**

Признаки, которые редко бывают одновременно ненулевыми на одном объекте, заменяются на один признак.

④ **Поиск порогов по гистограммам**

- Дискретизация признаков на бины
- Рассмотрение только границ бинов как кандидатов в пороги

**Вопрос:** В каких задачах LightGBM будет справляться наилучшим образом, с учётом таких особенностей?

Обучение на очень больших наборах данных

# Мотивация CatBoost

## Две ключевые проблемы:

- **Переобучение (смещённость) в градиентах:**  
 $e_i = \nabla \ell(a_{t-1}(x^i), y^i)$  вычисляются на тех же точках  $x^i$ , по которым обучался ансамбль  $a_{t-1}(x)$ .
- **Обработка категориальных признаков:**
  - Признаки с большим числом редких значений (пользователи, регионы, товары и т.д.)
  - Классические подходы (one-hot encoding) неэффективны

**Приём, похожий на Out-of-bag и на онлайнные методы:** Для получения несмещенных оценок для каждого  $x^i$  храним и достраиваем ансамбль на выборке без этого объекта

**Вопросы:** Как сделать, чтобы этих выборок было не слишком много?  
Как сделать, чтобы они не сильно пересекались?

# Упорядоченный бустинг (ordered boosting)

## Ключевые идеи:

- Вычислять градиент  $e^i$  по модели  $a_{t-1}$ , которая не обучалась на  $x^i$
- Использовать обучающие подвыборки увеличивающейся длины ( $2^j$  объектов)
- Строить множество случайных перестановок выборки

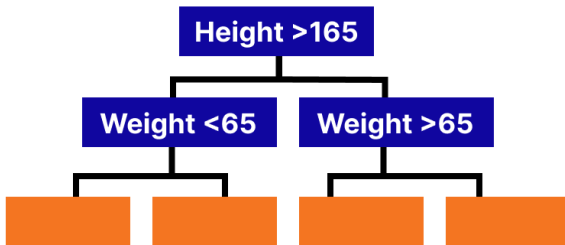
## Формальные обозначения:

- $\sigma_1, \dots, \sigma_s$  — случайные перестановки выборки  $X^n$
- $X^i$  — подвыборка первых  $2^j$  объектов из  $\sigma_r(X^n)$
- $a_t^i$  — модель, обученная на  $X^i$
- $e_{ti} = \nabla \ell(a_{t-1}^i(x^i), y^i)$  — градиент для  $(x_i, y_i)$
- $j = \lfloor \log_2(i - 1) \rfloor$  — определяет размер подвыборки

# Структура дерева в CatBoost

Дерево строится по уровням.

Для всех вершин одного уровня предикат одинаковый.

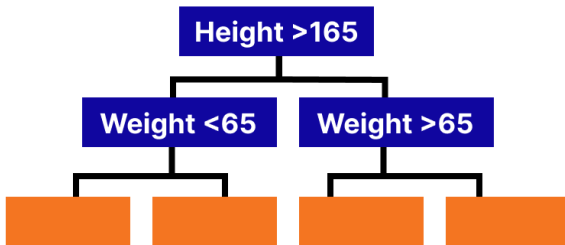


Вопрос: Чем может быть полезна такая структура дерева?

# Структура дерева в CatBoost

Дерево строится по уровням.

Для всех вершин одного уровня предикат одинаковый.



**Вопрос:** Чем может быть полезна такая структура дерева?

- Сильно снижается время обучения и предсказания.
- Уменьшается требуемая память.
- Является неплохой регуляризацией

# Модификация градиентного бустинга

- ① Сгенерировать случайные перестановки  $\sigma_0, \sigma_1, \dots, \sigma_s$
- ② Для всех  $t = 1, \dots, T$ :
  - ① Выбрать перестановку  $\sigma_r$  случайно из  $\sigma_1, \dots, \sigma_s$
  - ② Вычислить несмещённый вектор градиента  $e_{ti}, i = 1, \dots, n_I$
  - ③  $g_t := \operatorname{argmin}_{g \in \mathcal{G}} \sum_{i=1}^{n_I} (g(x^i) + e_{ti})^2$ , где в критерии ветвления  
слагаемые объектов  $x^i$  вычисляется по  $X^{rj}$
  - ④ Для всех деревьев  $g_t^{rj}$ :
    - Скопировать структуру дерева из  $g_t$
    - Вычислить значения в листьях по  $X^{rj}$
  - ⑤ Вычислить значения в листьях для  $g_t$  по  $X^{0j}$
  - ⑥ Вычислить  $\alpha_t$  и обновить  $f_i := f_i + \alpha_t g_t(x^i)$

# Способы обработки категориальных признаков

Пусть  $V$  — множество (словарь) значений признака  $f(x)$

Стандартные методы либо громоздки, либо переобучаются:

- Бинаризация (one-hot encoding):  $b_v(x) = [f(x) = v]$
- Группирование (кластеризация) значений (LightGBM)
- Статистика по целевому признаку (target statistics, TS):

$$\hat{f}(x) = \frac{\sum_{i=1}^{\ell} [f(x_i) = f(x)] y_i + \gamma p}{\sum_{i=1}^{\ell} [f(x_i) = f(x)] + \gamma}$$

CatBoost:

- Статистика TS также вычисляется по перестановкам  $X^{rj}$ :

$$\tilde{f}(x) = \frac{\sum_{x_i \in X^{rj}} [f(x_i) = f(x)] y_i + \gamma p}{\sum_{x_i \in X^{rj}} [f(x_i) = f(x)] + \gamma}$$

- Конъюнкции категориальных признаков создаются «налету» в процессе построения деревьев.

# Сравнение методов

|                         | XGBoost                                                      | LightGBM                                            | CatBoost                                                                    |
|-------------------------|--------------------------------------------------------------|-----------------------------------------------------|-----------------------------------------------------------------------------|
| Построение деревьев     | По уровням<br>Level-wise                                     | По листьям<br>Leaf-wise                             | По уровням<br>однородно                                                     |
| Поиск расщеплений       | Полный перебор;<br>Позже добавили<br>гистограммный<br>подход | Гистограммный<br>подход<br>(Использование<br>бинов) |                                                                             |
| Подвыборка              | subsample                                                    | -                                                   | feature_fraction                                                            |
| Категориальные признаки | -                                                            | +                                                   | +<br>Smoothed target<br>encoding;<br>Жадные<br>комбинации<br>кат. признаков |